

Guia de Defensa

Papaws

Sistema de microservicios distribuidos (.NET + Cassandra + React)

Como usar esta guia

Esta guia esta en formato pregunta -> respuesta. Cada tema tiene primero una explicacion clara del concepto y despues las preguntas que te pueden hacer, con la respuesta lista para decir y el lugar exacto del codigo donde se ve. Buscala por seccion o por palabra clave.

Estructura de cada bloque:

- P: la pregunta probable del docente.
- Respuesta: lo que conviene responder, explicado en simple.
- Donde mirar: archivo y metodo para mostrarlo en vivo si lo piden.

Indice de temas

- 1. La arquitectura en una frase
- 2. Los tres flujos clave (quien llama a quien)
- 3. Modelado en Cassandra (query-first)
- 4. Trazabilidad e historial (eventos)
- 5. Integridad entre servicios (sin foreign keys)
- 6. Concurrencia: el stock de productos (LWT)
- 7. Gastos y precios (refugio sin fines de lucro)
- 8. Refugio y cupo de rescatistas
- 9. Seguridad: JWT y roles
- 10. Reglas de negocio
- 11. Paginacion
- 12. Reportes (el servicio agregador)
- 13. Ver las queries reales en Cassandra
- 14. Resumen: quien llama a quien
- 15. Preguntas trampa y como salir bien

1. La arquitectura en una frase

Papaws son TRES microservicios .NET independientes, cada uno con su PROPIA base de datos Cassandra (keyspace propio), mas un frontend React que los consume. Ningun servicio entra a la base de otro: si necesita datos ajenos, los pide por la API REST.

- Animales (puerto 8080): rescatistas, organizaciones y animales. Es el dueño de esos datos. También hace el login y emite el JWT.
- Consulta (puerto 8081): veterinarios, servicios, productos (inventario) y consultas clínicas.
- Reportes (puerto 8082): NO tiene datos propios. Es un AGREGADOR: arma los reportes pidiendo datos por HTTP a los otros dos.
- Front (Pawpaws-Front, React + Vite): decide a que servicio ir por una 'zona' (animales / consulta) o la base de reportes.

P: Por que microservicios y no un monolito?

Para desacoplar dominios: cada servicio se despliega, escala y falla por separado. El precio a pagar es que no hay JOINS ni transacciones entre servicios; la consistencia se cuida a mano al escribir (sección 5). Es la decisión central a defender.

P: Como sabe el front a que microservicio pegar?

Cada llamada declara su 'zona'. La función `baseFor()` traduce la zona a la URL del servicio. Los endpoints concretos (que ruta, que metodo) estan centralizados en un solo archivo.

Donde mirar: `Pawpaws-Front/src/api/client.ts` (`baseFor` / `apiGet` / `apiPost`) y `Pawpaws-Front/src/api/endpoints.ts`

P: La regla de oro de la arquitectura?

Un servicio NUNCA lee ni escribe la base de datos de otro. Toda dependencia cruzada pasa por la API REST del dueño, reenviando el token JWT del usuario.

2. Los tres flujos clave (quien llama a quien)

Estas son las tres preguntas de recorrido 'de punta a punta' mas probables. Conviene poder trazarlas: front -> controlador -> servicio -> tabla de Cassandra.

2.1 Donde llaman los reportes a los rescatistas? (C1 / C2)

Recorrido:

- Front: en `Reportes.tsx` se elige el reporte y se llama `reportesApi.c1_rescatistaPorId(id)` o `c2_animalesPorRescatista(id)`. Pegan al servicio de REPORTES, no a Animales.
- Reportes: entra por `RescatistasReportesController.cs` y llama a `ReporteService` (`C1_RescatistaPorIdAsync` / `C2_AnimalesPorRescatistaAsync`).
- Aqui 'los reportes llaman a los rescatistas': `ReporteService` no tiene base; usa el cliente HTTP `AnimalesClient.GetRescatistaByIdAsync(id)` -> GET `api/rescatistas/{id}` contra Animales. Para C2 ademas `GetAnimalesByRescatistaAsync(id)` -> GET `api/animales/rescatista/{id}`.
- Animales responde desde `RescatistasController` -> `RescatistaService`, leyendo la tabla `rescatistas_by_id`.

Donde mirar: `Pawpaws-Reportes/Services/AnimalesClient.cs` y `ReporteService.cs`;
`Pawpaws-Animales/Services/RescatistaService.cs`

P: Y la autenticacion no se pierde al saltar de servicio?

No. El cliente de Reportes REENVIA el token JWT del usuario (copia el header `Authorization`), asi Animales sigue exigiendo login y rol. Ese reenvio esta en el metodo que arma el request del cliente.

Donde mirar: `Pawpaws-Reportes/Services/AnimalesClient.cs` (`CrearRequest copia Authorization`)

2.2 Donde llaman los rescatistas a los animales? (clic en 'Ver ficha')

- Front: el boton 'Ver ficha' en Rescatistas.tsx navega a /rescatistas/:id (ruta en App.tsx) y la pinta RescatistaDetalle.tsx.
- Aqui 'el rescatista llama a sus animales': RescatistaDetalle.tsx hace animalesApi.porRescatista(id) -> GET /api/animales/rescatista/{id} contra Animales.
- Animales: AnimalesController.ObtenerPorRescatista -> AnimalService.ObtenerPorRescatistaAsync, que lee la tabla animales_by_rescatista.

Donde mirar: Pawpaws-Front/src/pages/RescatistaDetalle.tsx; Pawpaws-Animales/Services/AnimalService.cs

P: Por que existe una tabla animales_by_rescatista aparte de animales_by_id?

Esta DESNORMALIZADA y PARTICIONADA por rescatista_id justo para que 'dame todos los animales de este rescatista' sea UNA sola lectura de una particion, sin JOIN ni scan. El mismo animal se guarda tambien en animales_by_id para buscarlo por su propio id. Ambas se escriben juntas en un BatchStatement para que no se desincronicen.

Donde mirar: Pawpaws-Animales/Data/CassandraSchema.cs y GuardarAnimalAsync en AnimalService.cs

2.3 Donde llaman los animales a sus consultas?

- Front: AnimalDetalle.tsx hace consultasApi.porAnimal(id) -> GET /api/consultas/animal/{id} contra el servicio de CONSULTA (las consultas viven en otro microservicio).
- Consulta: ConsultasController.ObtenerPorAnimal -> ConsultaService.ObtenerPorAnimalAsync.
- En Cassandra: primero lee la tabla indice consulta_codigos_by_animal (particionada por animal_id) para sacar los codigos; luego trae cada consulta de consultas_by_codigo. Es el patron 'tabla indice + lectura por clave'.

Donde mirar: Pawpaws-Front/src/pages/AnimalDetalle.tsx; Pawpaws-Consulta/Services/ConsultaService.cs

P: Si el animal y la consulta estan en servicios distintos, como se relacionan sin foreign key?

La consulta guarda el animal_id. No hay FK que cruce servicios; la relacion se sostiene por ese id y la consistencia se cuida al CREAR la consulta, validando el animal por HTTP (seccion 5).

3. Modelado en Cassandra (query-first)

Cassandra no hace JOINS ni filtra eficiente por columnas que no sean la clave de particion. Por eso se modela 'una tabla por consulta de lectura' (query-first) y el mismo dato se guarda varias veces (desnormalizacion). La clave de particion define donde vive el dato y por que campo se puede leer rapido.

Ejemplos en el sistema:

- Animales: animales_by_id (por id) y animales_by_rescatista (por rescatista).
- Consultas: consultas_by_codigo (por codigo); consulta_codigos_by_animal y consulta_codigos_by_veterinario (indices para listar); consulta_servicios_by_codigo y consulta_productos_by_codigo (relaciones de la consulta).

Donde mirar: Pawpaws-Animales/Data/CassandraSchema.cs y Pawpaws-Consulta/Data/CassandraSchema.cs

P: Y si quiero saber en que consultas se uso un servicio? (consulta inversa)

No se puede con consulta_servicios_by_codigo, porque esa tabla esta particionada por codigo (solo responde 'que servicios tiene ESTA consulta'). Por eso se agrego una tabla de INDICE INVERSO, consulta_codigos_by_servicio, particionada por servicio_id, que se mantiene sincronizada al crear, editar y borrar una consulta. La usa la ficha de servicio.

Donde mirar: ObtenerPorServicioAsync y los INSERT/DELETE del indice en Pawpaws-Consulta/Services/ConsultaService.cs

P: No es peligroso duplicar datos?

El riesgo es la desincronización. Se controla escribiendo TODAS las copias en la misma operación (BatchStatement) para que entren juntas, y manteniendo los índices en cada alta/baja/edición.

4. Trazabilidad e historial (eventos inmutables)

Para no perder historia, los cambios se guardan como EVENTOS INMUTABLES en tablas ordenadas por fecha descendente (clustering). En vez de pisar un valor, se agrega un evento nuevo.

- `eventos_custodia_by_animal`: ingreso del animal y cada reasignación entre rescatistas.
- `eventos_organizacion_by_rescatista`: alta y cambios de organización del rescatista.
- `eventos_adopcion_by_animal`: adopciones y devoluciones.

Donde mirar: Tablas y backfill en `Pawpaws-Animales/Data/CassandraSchema.cs`; registro en `AnimalService.cs` y `RescatistaService.cs`; líneas de tiempo en `AnimalDetalle.tsx` y `RescatistaDetalle.tsx`

P: Por qué el evento guarda el nombre y no solo el id?

Para que el historial siga siendo legible aunque después se de de baja ese rescatista u organización. Si guardara solo el id, al borrarse quedaría un id muerto sin sentido. Es un 'snapshot' del nombre en el momento del evento.

P: ¿Qué es el backfill?

Un proceso idempotente que siembra los eventos iniciales para los datos que ya existían antes de tener el historial, sin duplicar si se corre dos veces.

5. Integridad entre servicios (sin foreign keys)

Como cada servicio tiene su base, no hay claves foráneas que crucen servicios. La consistencia se cuida al ESCRIBIR:

- Al crear una consulta, Consulta valida el animal LLAMANDO POR HTTP a Animales (con reintentos). Si el animal no existe, rechaza.
- Esa misma validación trae el ESTADO del animal y rechaza agendar si está 'Adoptado' (un adoptado no recibe consultas hasta que se registre su devolución).

Donde mirar: `Pawpaws-Consulta/Services/AnimalReferenceService.cs` (`ObtenerEstadoAsync`) usado en `CrearAsync` de `ConsultaService.cs`

P: ¿Y si el servicio de Animales está caído cuando creo la consulta?

El cliente reintenta con backoff. Si aun así no hay respuesta, la creación FALLA con un error controlado, en lugar de crear una consulta que apunte a un animal que no se pudo verificar. Se prefiere fallar a dejar datos inconsistentes.

P: ¿Cómo se borra un animal sin dejar consultas huérfanas?

El animal se borra físicamente y, en cascada, se borran sus consultas. Si la baja de la cascada falla, se propaga el error y el animal NO se borra, para no dejar consultas apuntando a un animal inexistente.

Donde mirar: `EliminarAsync` en `Pawpaws-Animales/Services/AnimalService.cs` (cascada cross-servicio)

6. Concurrencia: el stock de productos (LWT)

Si dos consultas descuentan stock del mismo producto a la vez, un UPDATE normal podría pisar el valor del otro (lost update). Se resolvió con COMPARE-AND-SET usando LWT (transacciones ligeras) de Cassandra: UPDATE ... IF stock_disponible = ?. Si otro proceso cambió el stock entretanto, el IF falla, se lee el valor real que devuelve Cassandra y se reintenta.

Donde mirar: `AjustarStockAsync` en `Pawpaws-Consulta/Services/ProductoService.cs`

P: Por que no usar solo un UPDATE comun?

Porque dos UPDATE concurrentes no se enteran uno del otro: el ultimo gana y se pierde un descuento. El LWT agrega una condicion (IF) que solo aplica si el valor sigue siendo el que lei; si cambio, reintento con el valor nuevo. Es control de concurrencia optimista.

P: Es caro el LWT?

Si, mas que un write normal porque usa consenso (Paxos) entre replicas. Se justifica solo donde hay riesgo real de carrera, como el stock. No se usa para todo.

7. Gastos y precios (refugio sin fines de lucro)

El reporte de gastos lo calcula EL SERVICIO DE CONSULTA, porque ahi viven los precios: precio_base de los servicios y costo_unitario de los productos. Por cada consulta suma el costo de sus servicios mas el de los productos consumidos (costo_unitario x cantidad). El front lo agrupa por mes.

- Endpoint: GET /api/consultas/gastos (ObtenerGastosAsync en ConsultaService.cs).
- Las consultas canceladas se EXCLUYEN: devuelven el stock y no representan gasto real.
- Los precios son en bolivianos (Bs) y a precio de costo, porque es un refugio sin fines de lucro: no hay margen comercial.

Donde mirar: *Pawpaws-Consulta/Services/ConsultaService.cs (ObtenerGastosAsync);*
Pawpaws-Front/src/pages/Gastos.tsx

P: Por que el calculo de gastos no esta en Reportes?

Porque todos los datos necesarios (precio de servicios y costo de productos) viven en Consulta. Calcularlo ahi evita que Reportes tenga que pedir y recombinar todo por HTTP: se hace donde estan los datos.

P: Por que los numeros se muestran sin punto de miles?

Decision de formato: el punto se reserva para decimales. La moneda se muestra como 'Bs' con el locale es-BO y agrupacion desactivada, para que no se confunda 1.000 (mil) con 1.000 (uno coma cero).

8. Refugio y cupo de rescatistas

Existe un rescatista interno llamado 'Refugio' (oculto de la gestion) que actua como destino institucional de los animales. Cada rescatista normal tiene un CUPO MAXIMO de animales; el Refugio no tiene limite.

- Al dar de baja un rescatista, sus animales se reasignan: llenan el cupo del rescatista elegido y lo que sobra se deriva automaticamente al Refugio.
- Al reasignar un animal suelto, se bloquea si el rescatista destino ya esta en su cupo. Asi se reparten los animales del Refugio sin sobrecargar a nadie.

Donde mirar: *ReasignarAnimalesAsync y la validacion de cupo en ActualizarAsync de*
Pawpaws-Animales/Services/AnimalService.cs; constante CapacidadMaxima en Models/Rescatista.cs

P: Por que no asignar todos los animales a un solo rescatista al dar de baja?

Porque un rescatista no puede hacerse cargo de mas animales de los que su cupo permite. El sistema respeta ese limite y manda el excedente al Refugio, desde donde luego se reparten de a poco a quienes tengan lugar.

9. Seguridad: JWT y roles

Login con JWT y autorizacion por ROLES (Administrador, Encargado de consultas, Encargado de rescatistas). Cada endpoint se protege con [Authorize(Roles = ...)]. Cuando un servicio llama a otro, REENVIA el token del usuario, asi la autorizacion se respeta en toda la cadena.

- El secreto de firma del JWT y la contraseña NO estan en el repositorio: se leen de variables de entorno / user-secrets, y appsettings.json queda en blanco (fail-fast si faltan).
- La misma clave de firma es identica en los tres servicios: Animales firma, los demas validan.

Donde mirar: Carpetas Security/ de cada servicio; emision del token en Pawpaws-Animales (AuthController / JwtService)

P: Como respeta Reportes los permisos si no tiene los usuarios?

No necesita la base de usuarios: confia en el JWT firmado. Valida la firma con la clave compartida y lee los roles del token. Ademas reenvia ese token a Animales/Consulta, que vuelven a validar.

10. Reglas de negocio

Ejemplos concretos que conviene tener a mano:

- Una consulta no puede nacer 'Completada'.
- Un animal 'Adoptado' solo puede pasar a 'Devuelto al refugio', no directo a 'Disponible'.
- No se permiten productos (ni organizaciones) con nombre repetido entre los activos.
- La fecha de ingreso de un animal no puede ser futura.
- Un rescatista no puede superar su cupo de animales (seccion 8).

P: Borrado fisico o logico?

Rescatistas, veterinarios, servicios, productos y organizaciones usan BORRADO LOGICO (activo = false) para no romper referencias historicas. Los animales SI se borran fisicamente, y al hacerlo se borran en cascada sus consultas.

11. Paginacion

Hay paginacion en dos niveles. En el BACKEND, los listados aceptan ?pagina= y ?tamano= y devuelven un objeto con los items de esa pagina. En el FRONT, ademas, las listas largas se paginan del lado del cliente: 10 elementos por pagina con controles numerados (1, 2, 3 ...).

- Backend: helper Paginar() y un tamano por defecto; el front pide la primera pagina con tamano alto.
- Front: hook usePaginated y componente Pagination, aplicados a animales, productos, rescatistas, servicios, veterinarios, organizaciones, consultas y gastos.

Donde mirar: apiList en Pawpaws-Front/src/api/client.ts; Pawpaws-Front/src/hooks/usePaginated.ts y components/Pagination.tsx; Common/Paginacion.cs en cada servicio

P: Por que paginar tambien en el front si el backend ya pagina?

Porque el front trae la lista y la filtra/ordena en memoria para una UX fluida; paginar ahi evita renderizar listas gigantes de cientos de tarjetas. Son dos optimizaciones que se complementan.

12. Reportes (el servicio agregador)

Reportes no tiene base propia: cada reporte combina llamadas HTTP a Animales y/o Consulta. El ejemplo mas completo es C20 ('Rescatistas y animales por organizacion'): pide la organizacion, luego sus rescatistas, y para CADA rescatista sus animales; el 'join' organizacion -> rescatistas -> animales se arma EN MEMORIA en el servicio de Reportes, porque los datos viven en servicios distintos.

- Un reporte util de rendimiento es el ranking de veterinarios por cantidad de consultas (C22), paginado.

Donde mirar: Pawpaws-Reportes/Services/ReporteService.cs (C20_OrganizacionDetalleAsync, C22_VeterinariosPorConsultasAsync)

P: Y los datos huérfanos en los reportes?

Se resuelven los nombres a prueba de huérfanos: si una consulta apunta a un veterinario dado de baja se muestra '(dado de baja)', y si ya no existe, '(eliminado)', nunca un GUID suelto.

Donde mirar: *ResolverNombresVetAsync / ResolverNombresAnimalAsync en ReporteService.cs*

P: Por que el listado de consultas no trae servicios ni productos?

Para evitar un problema N+1 caro en una lista. El listado es liviano; los servicios y productos se cargan solo al abrir el detalle de una consulta.

Donde mirar: *ObtenerPorCodigoAsync en Pawpaws-Consulta/Services/ConsultaService.cs*

13. Ver las queries reales en Cassandra

Cassandra corre en dos contenedores Docker: papaws-cassandra-animales (keyspace papaws_animales) y papaws-cassandra-consulta (keyspace papaws_consulta). Cassandra NO loguea cada query por defecto; para 'verlas' se activa TRACING.

Forma A - Una query puntual (cqlsh)

docker exec -it papaws-cassandra-animales cqlsh

USE papaws_animales; TRACING ON; SELECT * FROM animales_by_rescatista WHERE rescatista_id = <uuid>;

Forma B - Capturar TODAS las queries (la que mas sirve)

- Activar al 100%: `docker exec papaws-cassandra-animales nodetool settraceprobability 1`
- Usar la app (abrir una ficha, generar un reporte).
- Leer: `docker exec papaws-cassandra-animales cqlsh -e "SELECT started_at, request, parameters FROM system_traces.sessions LIMIT 50;"` (la columna parameters trae el CQL real).
- APAGARLO al terminar (al 100% es caro): `nodetool settraceprobability 0`

Forma C - Inspeccionar datos directos (demostrar la desnormalizacion)

- `docker exec papaws-cassandra-animales cqlsh -e "SELECT * FROM papaws_animales.animales_by_rescatista LIMIT 20;"`
- `DESCRIBE KEYSPACE papaws_animales;` -> muestra todas las tablas con su clave de particion y clustering.

Forma D - Logs

- `docker logs -f papaws-animales` (requests HTTP que llegan al microservicio .NET).
- Cruzado con la Forma B muestra el camino completo: request HTTP -> servicio -> query a Cassandra.

14. Resumen: quien llama a quien

- Front -> Animales: rescatistas, organizaciones, animales y sus historiales.
- Front -> Consulta: veterinarios, servicios, productos, consultas y gastos.
- Front -> Reportes: todos los reportes.
- Consulta -> Animales (HTTP): solo para validar el animal y su estado al crear una consulta.
- Reportes -> Animales y Reportes -> Consulta (HTTP): para armar cada reporte combinando datos.
- Ningun servicio entra a la base de otro: siempre por la API REST, con el token JWT reenviado.

15. Preguntas trampa y como salir bien

P: Esto es realmente distribuido o son tres apps cualquiera?

Es distribuido: tres servicios con bases independientes, comunicados por red (HTTP/REST), con fallas y consistencia gestionadas explícitamente (reintentos, validación al escribir, eventos). No comparten base ni memoria.

P: Que pasa si dos servicios se desincronizan (un id que ya no existe)?

Se asume y se maneja: los reportes resuelven nombres a prueba de huérfanos, y las escrituras críticas validan antes de crear. La consistencia es eventual y defendida en cada punto de escritura.

P: Por que Cassandra y no PostgreSQL?

Por escala de escritura y disponibilidad: Cassandra reparte por particiones y no depende de un único nodo. El costo es modelar query-first y desnormalizar, que es justamente lo que mostramos.

P: Donde esta el punto unico de falla?

Se evita: cada servicio y cada base son independientes. Si Animales cae, Consulta no puede crear consultas nuevas (porque valida el animal) pero el resto sigue; la caída se traduce en un error controlado, no en datos corruptos.

P: Como demuestro en vivo que un servicio no toca la base de otro?

Mostrando que Reportes no tiene cadena de conexión a ninguna base (solo clientes HTTP) y que cada keyspace solo lo abre su propio servicio. Con la Forma B del tracing se ve que las queries de `papaws_consulta` solo salen del servicio de Consulta.